

Autonomous Vision & Behaviour Understanding

1. What the Project Does

Project Overview

Autonomous Vision & Behaviour Understanding is a computer-vision system that analyzes video footage to identify objects/persons, track them across frames, understand their actions, and detect unusual behaviour.

Unlike a simple object-detection system that only answers "**Is there a person?**", our system answers:

- **Who/which object is being tracked?**
- **Where is it?**
- **What is it doing?**
- **How long has it been doing it?**
- **Is the behaviour normal or abnormal?**
- **When did the unusual event occur?**

Selected Scenario: Workplace Safety

The system is designed for a **warehouse/workplace safety environment**.

For example:

A person walking through a warehouse is considered normal behaviour.

A person remaining stationary in the same location for an unusually long period, such as 10 minutes, can be flagged as abnormal.

The system processes CCTV/video input and performs the following pipeline:

Video Input

↓

Person/Object Detection

↓

Object Tracking

↓

Movement & Behaviour Analysis

↓

Normal / Abnormal Classification



Event Generation



Alert + Timestamp + Track ID

Example Output

Person ID: 07

Behaviour: Standing Still

Duration: 10 minutes 12 seconds

Status: ABNORMAL

Time: 00:14:32

Location: Warehouse Zone B

This ensures that every abnormal event is connected to a **specific entity and a specific time**.

2. Technologies, Libraries, and Models Used

Technologies

Technology Purpose

Python	Main programming language
OpenCV	Video processing and frame handling
YOLO	Person/object detection
ByteTrack	Tracking objects across frames
NumPy	Numerical calculations
Pandas	Event/result processing
PyTorch	Deep-learning model execution
Matplotlib	Result visualization
Streamlit	Optional web-based dashboard

Object Detection

We use a **YOLO (You Only Look Once)** model to detect people and other relevant objects in each video frame.

Example:

Frame

↓

YOLO

↓

Person detected

↓

Bounding box + confidence score

The detector provides:

- Object class
- Bounding-box coordinates
- Confidence score

For example:

Person

Confidence: 0.94

Bounding Box: (120, 80, 320, 500)

Object Tracking

Detection alone cannot determine whether the person in the current frame is the same person from the previous frame.

Therefore, a tracker such as **ByteTrack** is used.

Example:

Frame 1 → Person ID 01

Frame 2 → Person ID 01

Frame 3 → Person ID 01

Frame 4 → Person ID 01

This allows the system to calculate:

- Movement
- Speed
- Direction
- Time spent in an area
- Stationary duration
- Entry/exit events

Behaviour Analysis

The system calculates behavioural features from the tracked objects.

For example:

Position change < threshold

+

Time stationary > threshold

↓

Standing/Stationary Behaviour

The system can classify behaviour such as:

- Walking
- Standing
- Moving away
- Moving toward
- Entering a zone
- Leaving a zone
- Remaining stationary

Anomaly rules are then applied to identify unusual behaviour.

3. How to Install Dependencies

Requirements

Recommended environment:

- Python 3.10+

- Windows/Linux/macOS
- 8 GB+ RAM
- NVIDIA GPU recommended for faster inference
- Webcam or recorded video

Clone the Repository

```
git clone <YOUR_GITHUB_REPOSITORY_URL>
```

```
cd autonomous-vision-behaviour
```

Create a Virtual Environment

Windows

```
python -m venv venv
```

```
venv\Scripts\activate
```

Linux/macOS

```
python3 -m venv venv
```

```
source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Example requirements.txt:

```
ultralytics
```

```
opencv-python
```

```
numpy
```

```
pandas
```

```
torch
```

```
torchvision
```

```
matplotlib
```

```
streamlit
```

If using a GPU, install the appropriate PyTorch/CUDA version for the target machine.

4. How to Configure and Run the System

Project Structure

A recommended repository structure is:

autonomous-vision-behaviour/

```
|
|
|└─ data/
|
|  └─ input/
|  └─ output/
|
|
|└─ models/
|  └─ yolo_model.pt
|
|
|└─ src/
|  └─ detector.py
|  └─ tracker.py
|  └─ behaviour.py
|  └─ anomaly.py
|  └─ main.py
|
|
|└─ results/
|
|
|└─ requirements.txt
|
|└─ config.yaml
└─ README.md
```

Configuration

The behaviour thresholds can be stored in config.yaml.

Example:

model: models/yolo_model.pt

confidence_threshold: 0.5

stationary:

movement_threshold: 20

duration_seconds: 600

output:

save_video: true

save_events: true

Here:

600 seconds = 10 minutes

means a person remaining stationary for approximately 10 minutes will be considered unusual.

Run the System

Place the input video inside:

data/input/

Then run:

```
python src/main.py --input data/input/test_video.mp4
```

The system will:

1. Read the video.
2. Detect people/objects.
3. Assign tracking IDs.
4. Track their movement.
5. Calculate behavioural features.
6. Identify normal/abnormal behaviour.
7. Record the timestamp of abnormal events.
8. Generate an annotated output video.

Example:

Input:

data/input/warehouse.mp4

Output:

data/output/warehouse_result.mp4

data/output/events.csv

5. How to Reproduce the Demonstrated Results

To reproduce the results shown in the project demonstration:

Step 1 — Prepare the Video

Place the demonstration video in:

data/input/demo.mp4

The video should contain people moving through a workplace/warehouse environment.

Step 2 — Set the Configuration

Use:

confidence_threshold: 0.5

stationary:

movement_threshold: 20

duration_seconds: 600

Step 3 — Run the Model

```
python src/main.py --input data/input/demo.mp4
```

Step 4 — Check the Output

The processed video will contain bounding boxes and tracking IDs:

Person ID: 03	
Behaviour: WALKING	

Status: NORMAL

If a person remains stationary beyond the configured threshold:

Person ID: 07
Behaviour: STANDING
Duration: 10:02
Status: ABNORMAL
Time: 00:14:32

Step 5 — Check the Event Log

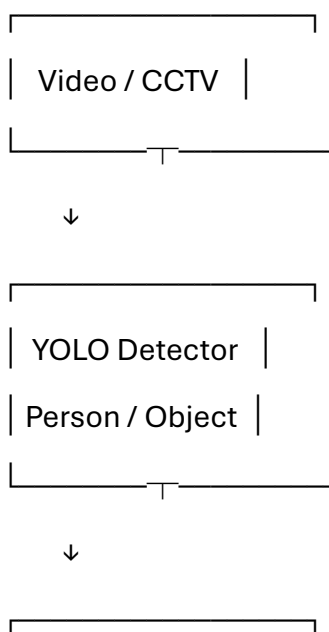
The system produces an event file such as:

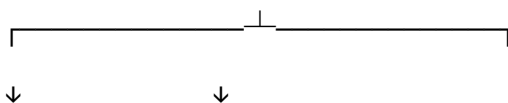
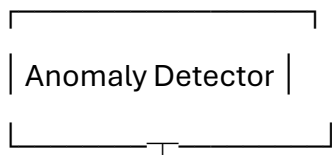
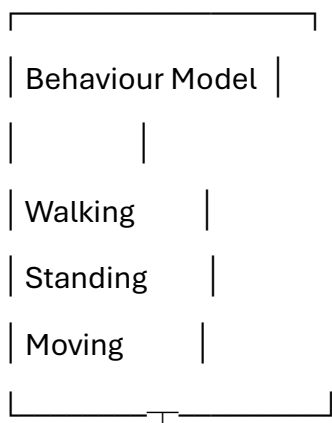
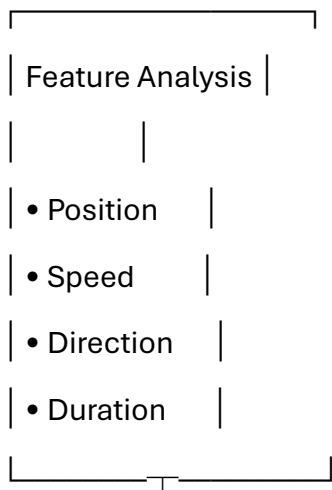
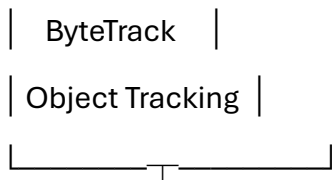
track_id,behaviour,start_time,end_time,duration,status

7,standing,00:04:30,00:14:32,602,abnormal

This makes the result reproducible and demonstrates **which entity behaved unusually and exactly when it happened.**

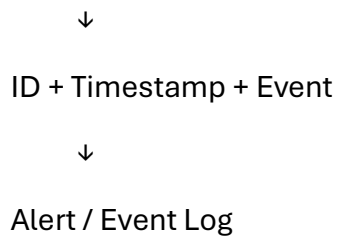
System Architecture





NORMAL BEHAVIOUR

ABNORMAL BEHAVIOUR



Why This Meets the Challenge Requirements

The project does **more than object detection**.

Challenge Requirement	Our System
Recognize what people/objects are doing	Behaviour analysis
Track the same object	ByteTrack IDs
Find objects accurately	YOLO detection
Detect meaningful events	Behaviour + anomaly rules
Distinguish normal/abnormal behaviour	Threshold-based anomaly detection
Identify who and when	Track ID + timestamp
Handle multiple objects	Multi-object tracking
Reproduce results	Fixed configuration + input video + event CSV